Universidad de Santander UDES | ACREDITACIÓN INTERNACIONAL EQUAA Education Quality Accreditation Agency

Patrones de Diseño de Software

Unidad 7: Patrones Arquitectónicos I

Post-Contenido 2

Ingeniería de Sistemas

2026

Objetivo de la Actividad

El estudiante implementa un módulo de gestión de productos aplicando arquitectura hexagonal (Ports & Adapters), definiendo puertos de entrada y salida como interfaces Java puras e implementando adaptadores REST y JPA que no contaminen el dominio con detalles técnicos.

Prerrequisitos

Requisito	Versión / Detalle
Java JDK	17 o superior
Spring Boot	3.x con dependencias: Spring Web, Spring Data JPA, H2 Database
Maven	3.8+
Post-Contenido 1	Haber completado el laboratorio anterior (arquitectura en capas)
Conocimientos previos	Interfaces Java, inyección de dependencias con Spring, JPA básico
Herramienta de pruebas	curl o Postman

Laboratorio: Módulo de Productos con Arquitectura Hexagonal

Paso 1 — Estructura de Paquetes Hexagonal

La estructura hexagonal separa estrictamente el dominio de la infraestructura:

```
com.example.hexagonal/
├── domain/
│   ├── model/
│   │   └── Producto.java          ← Entidad de dominio (sin anotaciones JPA)
│   ├── port/
│   │   ├── in/                    ← Puertos de entrada (casos de uso)
│   │   │   ├── CrearProductoUseCase.java
│   │   │   ├── ListarProductosUseCase.java
│   │   │   ├── ActualizarStockUseCase.java
│   │   │   └── out/               ← Puertos de salida
│   │   │       └── ProductoRepositoryPort.java
│   │   └── service/
│   │       └── ProductoDomainService.java  ← Implementa puertos de entrada
├── adapter/
│   ├── in/
│   │   └── web/
│   │       └── ProductoController.java ← Adaptador REST (driving)
│   └── out/
│       └── persistence/
│           ├── ProductoJpaEntity.java   ← Entidad JPA (solo aquí)
│           ├── ProductoJpaRepository.java
│           └── ProductoRepositoryAdapter.java ← Implementa puerto de salida
├── config/
│   └── BeanConfiguration.java          ← Wiring de Spring
└── HexagonalApplication.java
```

Paso 2 — Dominio Puro: Modelo y Puertos

La entidad de dominio es un POJO Java sin anotaciones de frameworks externos:

```
// domain/model/Producto.java
public class Producto {
    private Long id;
    private String nombre;
    private String descripcion;
    private BigDecimal precio;
    private int stock;

    // Lógica de negocio pura en el dominio
    public void reducirStock(int cantidad) {
        if (cantidad > this.stock) {
            throw new StockInsuficienteException(
                "Stock insuficiente. Disponible: " + this.stock);
        }
        this.stock -= cantidad;
    }

    public boolean estaDisponible() {
        return this.stock > 0;
```

```
    }
    //          Constructores,          getters          y          setters
}
```

Los puertos de entrada definen los casos de uso del sistema:

```
//                                domain/port/in/CrearProductoUseCase.java
public            interface            CrearProductoUseCase            {
    Producto                  crear(Producto                  producto);
}

//                            domain/port/in/ListarProductosUseCase.java
public            interface            ListarProductosUseCase            {
    List<Producto>                                          listarTodos();
    Producto                    buscarPorId(Long                    id);
}

//                            domain/port/in/ActualizarStockUseCase.java
public            interface            ActualizarStockUseCase            {
    Producto       reducirStock(Long       id,       int       cantidad);
}
```

El puerto de salida define lo que el dominio necesita de la persistencia:

```
//                            domain/port/out/ProductoRepositoryPort.java
public            interface            ProductoRepositoryPort            {
    Producto                  guardar(Producto                  producto);
    Optional<Producto>                buscarPorId(Long                id);
    List<Producto>                                          buscarTodos();
    void                        eliminar(Long                        id);
}
```

Checkpoint de Verificación

✓ Las clases en domain/ no importan ninguna clase de Spring ni de JPA

✓ Los puertos (interfaces) están en domain/port/in/ y domain/port/out/

✓ La clase Producto compila sin dependencias externas (solo java.*)

Paso 3 — Servicio de Dominio: Implementa los Puertos de Entrada

El servicio de dominio implementa los casos de uso e inyecta el puerto de salida:

```
//                              domain/service/ProductoDomainService.java
public                    class                    ProductoDomainService
        implements                                 CrearProductoUseCase,
                   ListarProductosUseCase,
                   ActualizarStockUseCase                                {

    private       final       ProductoRepositoryPort       repositoryPort;

    public  ProductoDomainService(ProductoRepositoryPort  repositoryPort)
{
        this.repositoryPort              =              repositoryPort;
    }

    @Override
    public        Producto        crear(Producto        producto)        {
        if   (producto.getPrecio().compareTo(BigDecimal.ZERO)   <=   0)   {
```

```
            throw new PrecioInvalidoException("El precio debe ser mayor
a                                                                    cero");
        }
        return                          repositoryPort.guardar(producto);
    }

    @Override
    public            List<Producto>            listarTodos()            {
        return                              repositoryPort.buscarTodos();
    }

    @Override
    public         Producto         buscarPorId(Long         id)         {
        return                              repositoryPort.buscarPorId(id)
            .orElseThrow(() -> new ProductoNotFoundException("Producto "
+           id           +           "           no           encontrado"));
    }

    @Override
    public    Producto    reducirStock(Long    id,    int    cantidad)    {
        Producto           producto           =           buscarPorId(id);
        producto.reducirStock(cantidad);   //   lógica   en   el   dominio
        return                          repositoryPort.guardar(producto);
    }
}
```

Nota: ProductoDomainService NO tiene @Service de Spring. Es una clase Java pura que se registra en BeanConfiguration.

Paso 4 — Adaptador de Persistencia (Driven Adapter)

La entidad JPA vive únicamente en el adaptador de persistencia:

```
//                        adapter/out/persistence/ProductoJpaEntity.java
@Entity
@Table(name                        =                        "productos")
public              class              ProductoJpaEntity              {
    @Id     @GeneratedValue(strategy     =     GenerationType.IDENTITY)
    private                            Long                            id;
    private                          String                        nombre;
    private                       String                     descripcion;
    private                        BigDecimal                     precio;
    private                            int                          stock;
    //         Constructores,         getters         y         setters
}

//                adapter/out/persistence/ProductoRepositoryAdapter.java
@Component
public class ProductoRepositoryAdapter implements ProductoRepositoryPort
{

    private       final       ProductoJpaRepository       jpaRepository;

    public                ProductoRepositoryAdapter(ProductoJpaRepository
jpaRepository)                                                          {
        this.jpaRepository                =                jpaRepository;
    }

    @Override
```

```
    public Producto guardar(Producto producto) {
        ProductoJpaEntity entity = toEntity(producto);
        ProductoJpaEntity saved = jpaRepository.save(entity);
        return toDomain(saved);
    }

    @Override
    public Optional<Producto> buscarPorId(Long id) {
        return jpaRepository.findById(id).map(this::toDomain);
    }

    @Override
    public List<Producto> buscarTodos() {
        return jpaRepository.findAll().stream()
            .map(this::toDomain).collect(Collectors.toList());
    }

    @Override
    public void eliminar(Long id) {
        jpaRepository.deleteById(id);
    }

    private Producto toDomain(ProductoJpaEntity e) {
        return new Producto(e.getId(), e.getNombre(),
            e.getDescripcion(), e.getPrecio(), e.getStock());
    }

    private ProductoJpaEntity toEntity(Producto p) {
        return new ProductoJpaEntity(null, p.getNombre(),
            p.getDescripcion(), p.getPrecio(), p.getStock());
    }
}
```

Paso 5 — Adaptador Web (Driving Adapter)

El controlador REST actúa como adaptador primario: traduce HTTP en llamadas a los puertos de entrada:

```
// adapter/in/web/ProductoController.java
@RestController
@RequestMapping("/api/productos")
public class ProductoController {

    private final CrearProductoUseCase crearUseCase;
    private final ListarProductosUseCase listarUseCase;
    private final ActualizarStockUseCase stockUseCase;

    public ProductoController(CrearProductoUseCase crearUseCase,
                              ListarProductosUseCase listarUseCase,
                              ActualizarStockUseCase stockUseCase) {
        this.crearUseCase = crearUseCase;
        this.listarUseCase = listarUseCase;
        this.stockUseCase = stockUseCase;
    }

    @GetMapping
    public List<Producto> listar() {
        return listarUseCase.listarTodos();
```

```
    }

    @GetMapping("/{id}")
    public Producto buscar(@PathVariable Long id) {
        return listarUseCase.buscarPorId(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Producto crear(@RequestBody Producto producto) {
        return crearUseCase.crear(producto);
    }

    @PatchMapping("/{id}/stock")
    public Producto reducirStock(@PathVariable Long id,
                                 @RequestParam int cantidad) {
        return stockUseCase.reducirStock(id, cantidad);
    }
}
```

Paso 6 — Configuración de Beans (Wiring)

Spring no puede detectar ProductoDomainService con @Service porque es una clase Java pura. El wiring se realiza explícitamente en la clase de configuración:

```
// config/BeanConfiguration.java
@Configuration
public class BeanConfiguration {

    @Bean
    public ProductoDomainService productoDomainService(
            ProductoRepositoryPort repositoryPort) {
        return new ProductoDomainService(repositoryPort);
    }
}
```

Este archivo es el único punto donde Spring "conoce" el dominio. El dominio mismo no tiene anotaciones Spring — por eso puede probarse sin levantar ningún contexto.

Checkpoint de Verificación

✓ El proyecto compila con mvn clean package sin errores

✓ GET /api/productos retorna lista vacía (200 OK)

✓ POST /api/productos crea un producto y retorna 201 Created

✓ PATCH /api/productos/{id}/stock con cantidad mayor al stock retorna 400 con mensaje de error

✓ Las clases en domain/ no tienen ningún import de org.springframework ni javax.persistence

✓ ProductoDomainService se puede instanciar en una prueba JUnit sin @SpringBootTest

✓ El adaptador ProductoRepositoryAdapter traduce correctamente entre Producto y ProductoJpaEntity

Rúbrica de Evaluación

Criterio	Excelente (90–100%)	Bueno (70–89%)	Aceptable (50–69%)	Insuficiente (0–49%)
Implementación (40%)	El código implementa correctamente la arquitectura solicitada con separación de capas/puertos evidente; sigue convenciones Java y Spring Boot.	La implementación cubre la mayoría de los requisitos con separación de capas aceptable; algunos detalles menores incorrectos.	El código funciona parcialmente; la separación de capas es inconsistente o hay dependencias cruzadas no permitidas.	El código no compila, no implementa la arquitectura solicitada o mezcla capas sin justificación.
Funcionalidad (25%)	Todos los endpoints responden correctamente a las pruebas; los checkpoints están verificados y el sistema persiste datos correctamente.	La mayoría de los endpoints funcionan; algunos casos edge no están cubiertos pero la funcionalidad core es correcta.	Al menos la mitad de los endpoints funcionan; hay errores en la persistencia o en la respuesta HTTP.	Los endpoints no responden o retornan errores sistemáticos; la funcionalidad básica no está implementada.
Documentación (20%)	El README explica la arquitectura implementada con diagrama de capas/puertos, instrucciones de ejecución y al menos 3 capturas relevantes.	El README cubre los puntos principales con instrucciones de ejecución claras; el diagrama es básico pero correcto.	El README existe pero es incompleto; faltan instrucciones o el diagrama no refleja la implementación real.	Sin README o con información incorrecta que no corresponde a la implementación entregada.
Estilo y Convenciones (15%)	El código usa convenciones Java estándar (camelCase, PascalCase), tiene al menos 3 commits descriptivos y la estructura de paquetes es limpia.	Las convenciones se siguen en su mayoría; hay al menos 3 commits aunque algunos mensajes son genéricos.	Hay inconsistencias en convenciones; los commits son escasos o con mensajes no descriptivos.	El código ignora convenciones estándar; hay un solo commit o los mensajes no describen los cambios realizados.

Entregables

Elemento	Especificación
Plataforma	GitHub (repositorio público)
Nombre del repositorio	apellido-post2-u7
Estructura requerida	README.md + código fuente Maven + capturas de pantalla
Commits mínimos	Mínimo 3 commits con mensajes descriptivos del progreso
Capturas requeridas	Pruebas con curl o Postman mostrando respuestas HTTP exitosas